

ZYProject 核心模型与本地存储解析

ZYProject 核心模型与本地存储解析

一、ZYNetWorkModel 是干嘛的？

ZYNetWorkModel 是项目中 **统一处理网络请求返回数据的基础模型类**，核心作用是“翻译”服务器返回的 JSON 数据，转换为 iOS 代码可直接使用的对象，相当于网络数据的“统一适配器”。

1. 为什么要用它？

先看 ZYNetWorkModel 的结构与实现，核心是“统一数据格式”与“字段映射”：

(1) 类结构定义

```
@interface ZYNetWorkModel : NSObject
@property (strong, nonatomic) NSString *message; // 提示信息（如错误描述、成功提示）
@property (strong, nonatomic) NSString *status; // 请求状态码（如"1"成功、"2"失败、"3"登录失效）
@property (strong, nonatomic) id data; // 业务数据（服务器返回的核心数据，类型为id，适配不同业务）
@end
```

(2) 字段映射实现（YYModel 协议）

```
@implementation ZYNetWorkModel
// 自定义字段映射：解决服务器字段与模型属性名不一致问题
+ (NSDictionary *)modelCustomPropertyMapper {
    return @{
        @"message": @"message", // 服务器字段"message" → 模型属性"message"（名一致，可省略，此处为清晰保留）
        @"status": @"status", // 服务器字段"status" → 模型属性"status"
        @"data": @"t" // 服务器字段"t" → 模型属性"data"（核心映射：业务数据存于服务器）
    };
}
```

```
"t"字段)
    };
}
@end
```

关键说明：

服务器返回的 JSON 数据中，**业务核心数据存于“t”字段**，通过 `modelCustomPropertyMapper` 映射为模型的 `data` 属性，统一对外提供访问入口，避免业务层直接依赖服务器字段名。

2. 实际使用流程

所有网络请求成功后，均通过 `ZYNetWorkModel` 解析数据，逻辑统一且清晰，以 `ZYNetworkRequest.m` 为例：

```
// 网络请求成功回调
- (void)requestSuccessWithResponse:(id)response success:(void (^)(ZYNetWorkModel *))
success {
    // 1. JSON数据 → ZYNetWorkModel 对象（使用YYModel解析）
    ZYNetWorkModel *model = [ZYNetWorkModel yy_modelWithJSON:response];

    // 2. 根据 status 字段判断请求结果，执行对应逻辑
    if ([model.status isEqualToString:@"1"]) {
        // 状态1：请求成功 → 回调返回模型（业务层从 model.data 获取核心数据）
        if (success) {
            success(model);
        }
    } else if ([model.status isEqualToString:@"2"]) {
        // 状态2：业务错误 → 显示错误提示（从 model.message 获取描述）
        [EasyTextView showErrorText:model.message];
    } else if ([model.status isEqualToString:@"3"]) {
        // 状态3：登录失效 → 统一跳转登录页
        [self jumpToLoginViewController];
    } else {
        // 其他状态：默认错误提示
        [EasyTextView showErrorText:@"请求失败，请稍后重试"];
    }
}
```

核心优势

- **统一处理**：所有网络请求共用一套解析逻辑，避免重复代码；
- **结构清晰**：通过 `status` 判断结果、`data` 获取业务数据、`message` 获取提示，职责明确；
- **解耦**：网络层专注“数据解析与状态判断”，业务层仅需关注 `model.data` 中的具体数据，不依赖服务器字段结构。

二、具体模型的使用

项目中的 **业务模型**（如 `ZYCityArea`、`ZYNewRoomModel`、`SLCity`），用于解析 `ZYNetWorkModel.data` 中的核心业务数据，实现“通用模型→具体业务模型”的转换。

1. 使用示例：获取城市列表

以 `ZYMainViewController.m` 中“获取城市列表”为例，展示完整流程：

```
// 1. 构建请求参数与URL
BADataEntity *parameters = [BADataEntity new];
parameters.urlString = kcityCnListURL; // 城市列表接口URL
parameters.parameters = @{@"type": @"1"}; // 请求参数
// 2. 发起网络请求（使用postBeanNoPrompt方法，无成功提示）
[ZYNetworkRequest postBeanNoPrompt:parameters
    url:kcityCnListURL
    success:^(id json) {
        // 3. 解析为 ZYNetWorkModel
        ZYNetWorkModel *object = (ZYNetWorkModel *)json;
        if ([object.status isEqualToString:@"1"]) {
            // 4. 获取业务数据：model.data 为城市列表数组（NSArray）
            NSArray *dataArray = (NSArray *)object.data;

            // 5. 遍历数组，转换为具体业务模型 SLCity
            NSMutableArray *cityArrs = [NSMutableArray array];
            [dataArray enumerateObjectsUsingBlock:^(id _Nonnull obj, NSUInteger idx,
                BOOL * _Nonnull stop) {
                SLCity *cityModel = [SLCity new];
                // 赋值：从JSON对象（obj）中提取字段，赋值给业务模型属性
                cityModel.name = [obj objectForKey:@"name"]; // 城市名称
                cityModel.id = [obj objectForKey:@"sysAreaCode"]; // 城市编码
            }];
        }
    }];
```

```

        [cityArrs addObject:cityModel];
    }];

    // 6. 使用业务模型数组（cityArrs）更新UI（如刷新列表）
    self.cityList = cityArrs;
    [self.cityTableView reloadData];
}
}
failure:^(NSError *error) {
    // 请求失败处理
    [EasyTextView showErrorText:error.localizedDescription];
}];

```

2. 具体模型的字段映射

业务模型需实现 `modelCustomPropertyMapper` 方法（YYModel 协议），处理“服务器字段与模型属性名不一致”问题，以 `ZYCityArea` 为例：

```

@interface ZYCityArea : NSObject
@property (nonatomic, copy) NSString *name;    // 城市名称
@property (nonatomic, copy) NSString *areaName; // 区域名称
@property (nonatomic, copy) NSString *district; // 区县编码
@property (nonatomic, copy) NSString *businessareauuid; // 商圈UUID
@end

@implementation ZYCityArea
// 字段映射：服务器字段 → 模型属性
+ (NSDictionary *)modelCustomPropertyMapper {
    return @{
        @"name" : @"name",    // 名一致，直接映射
        @"areaName" : @"areaName", // 名一致，直接映射
        @"district" : @"district", // 名一致，直接映射
        @"businessareauuid" : @"businessareauuid" // 名一致，直接映射
    };
}
@end

```

说明：若服务器字段与模型属性名不同（如服务器“`area_code`” → 模型“`areaCode`”），需在映射字典中明确配置（`@"areaCode" : @"area_code"`），确保解析正确。

三、whc_SqliteVersion 是什么意思？

```
+ (NSString *)whc_SqliteVersion {  
    return @"1.0";  
}
```

该方法是项目 **本地数据库版本管理的核心**，依赖第三方库 WHC_ModelSqliteKit（一款基于 SQLite 的 ORM 框架，支持“模型→数据库表”自动映射）。

1. 核心作用

（1）版本标识

返回的字符串（如 @"1.0"）是 **业务模型对应的数据库表版本号**，用于标记“模型结构与数据库表结构的匹配关系”。

（2）表结构更新触发

当业务模型的属性发生变化（如新增字段、删除字段、修改字段类型）时，必须 **更新 whc_SqliteVersion 返回的版本号**（如从 @"1.0" 改为 @"1.1"），否则 WHC_ModelSqliteKit 无法识别表结构变更，导致数据存储异常。

（3）自动数据迁移

框架会对比“模型当前版本号”与“本地数据库表记录的版本号”：

- 若版本号一致：直接使用现有表结构；
- 若版本号不一致：自动执行“表结构更新”（如新增字段、删除废弃字段），并保留原有数据（前提是字段变更符合迁移规则）。

2. 框架内部逻辑（从 WHC_ModelSqlite.m 提取）

```
// 检查模型版本号，判断是否需要更新表结构  
- (void)checkModelVersionWithClass:(Class)model_class {  
    // 1. 获取模型的当前版本号（调用 whc_SqliteVersion 方法）  
    NSString *currentVersion = [self exceSelector:@selector(whc_SqliteVersion) modelClass:
```

```

model_class];
if (!currentVersion || currentVersion.length == 0) {
    currentVersion = @"1.0"; // 默认版本号
}

// 2. 获取本地数据库表记录的版本号
NSString *localModelName = [self localNameWithModel:model_class]; // 本地表名（含版本号）
NSString *localVersion = [self extractVersionFromLocalModelName:localModelName]; // 提取本地版本号

// 3. 版本号对比：不一致则更新表结构
if ([self sharedInstance].check_update && ![currentVersion isEqualToString:localVersion]) {
    // 执行表结构更新（新增/删除字段，保留数据）
    [self updateTableFieldWithModel:model_class
        newVersion:currentVersion
        localModelName:localModelName];
    // 更新本地表名中的版本号
    [self updateLocalModelName:model_class newVersion:currentVersion];
}
}
}

```

四、项目中的本地数据存储

项目通过 `WHC_ModelSqliteKit` 实现本地数据存储，核心是“ORM 映射”（Object-Relational Mapping，对象 - 关系映射），将业务模型直接映射为 SQLite 数据库表，无需手动编写 SQL 语句。

1. 存储核心特性

（1）基于 SQLite

底层使用 SQLite 数据库，支持本地持久化存储，数据不会因 App 重启丢失（除非主动删除）。

（2）自动建表

框架根据业务模型的属性，自动创建数据库表：

- 表名：默认格式为“模型类名_v 版本号”（如 `SLCity_v1.0`）；

- 字段：模型的每个属性对应表的一个字段，字段类型自动匹配（如 `NSString`→`TEXT`，`NSNumber`→`INTEGER`）。

(3) 版本管理

通过 `whc_SqliteVersion` 实现版本控制，支持表结构自动更新与数据迁移，无需手动处理版本兼容问题。

(4) 简化操作

提供简洁的 API，支持“增删改查”操作，无需编写 SQL，示例：

```
// 1. 存储单个模型对象
SLCity *cityModel = [SLCity new];
cityModel.name = @"北京";
cityModel.Id = @"110000";
[WHC_ModelSqlite insert:cityModel];
// 2. 查询所有城市
NSArray *allCities = [WHC_ModelSqlite queryAll:SLCity.class];
// 3. 根据条件查询（如查询ID为110000的城市）
NSArray *targetCity = [WHC_ModelSqlite query:SLCity.class where:@"Id = ?" params:@[@"110000"]];
// 4. 删除所有城市
[WHC_ModelSqlite deleteAll:SLCity.class];
```

2. 存储位置

默认存储在 App 沙盒的 **Library/Caches/WHCSqlite/** 目录下，每个业务模型对应一个 SQLite 文件，文件名格式为“模型类名_v 版本号.sqlite”（如 `SLCity_v1.0.sqlite`）。

路径获取逻辑（从 `WHC_ModelSqlite.m` 提取）

```
+ (NSString *)databaseCacheDirectory:(Class)model_class {
    // 1. 优先使用模型自定义的存储路径（若模型实现 whc_SqlitePath 方法）
    NSString *customPath = [self exceSelector:@selector(whc_SqlitePath) modelClass:model_class];
    if (customPath && customPath.length > 0) {
```

```
        return customPath;
    }

    // 2. 默认路径：沙盒 Library/Caches/WHCSqlite/
    return [NSString stringWithFormat:@"%s/Library/Caches/WHCSqlite/", NSHomeDirectory()];
}
```

说明：可通过在业务模型中实现 `whc_SqlitePath` 方法，自定义数据库文件的存储路径，满足特殊需求（如多账号数据隔离）。

（注：文档部分内容可能由 AI 生成）